CNN: 합성곱 신경망

합성곱 · Stride · Padding · Pooling, PyTorch 실습

목차

01 왜 CNN

02 합성곱

03 Stride와 Padding

04 출력 크기와 채널

05 ReLU와 Pooling

06 전체 구조

07 출력층과 손실 함수

08 학습하기

09 실습 ① 이진 분류

10 실습 ② 다중 분류

11 문제와 정리

이미지는 숫자 행렬

	흑백 이미지	컬러 이미지
구성	높이 × 너비	높이 × 너비 × 3채널 (R·G·B)
픽셀 값	밝기 숫자 1개 (0~255 → 학습 시 0~1)	채널마다 숫자 1개씩, 총 3개
예시	Fashion-MNIST: 28 × 28 = 784개	CIFAR-10: 32 × 32 × 3 = 3,072개
PyTorch 모양	`(1, 28, 28)` = (채널, 높이, 너비)	`(3, 32, 32)`

컬러 이미지 = 흑백 이미지 3장(R, G, B)을 겹친 것

지금까지 배운 신경망(MLP)의 한계

용어 FC층(Fully Connected Layer): 모든 뉴런이 서로 연결된 층 하나 · MLP(다층 퍼셉트론): FC층을 여러 개 쌓은 신경망

① 위치 정보 손실

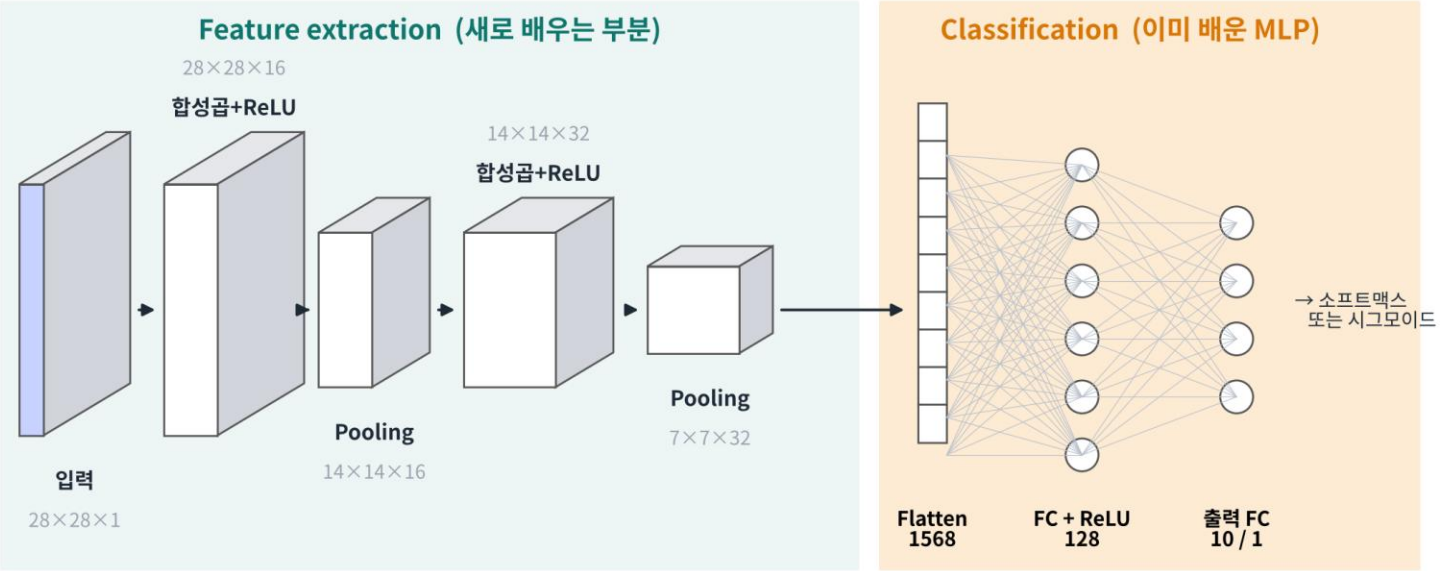
이미지를 한 줄로 펴서(Flatten) 입력해야 함. 바로 위아래 픽셀이 784칸 중 28칸 떨어진 자리로 가서 '가까이 있음' 정보가 사라짐.

② w 개수 폭증 (큰 이미지일 때)

224×224 컬러 이미지를 은닉 뉴런 1000개에 연결하면 w 약 1.5억 개. 학습이 느리고 과적합이 쉽게 생김.

→ 이미지를 모양 그대로 다루면서, 어느 위치의 같은 패턴이든 같은 커널로 찾는 방법이 필요함 = CNN

CNN 한눈에 보기



- Feature extraction: 이미지 모양 그대로 특징 추출. 가로·세로는 작아지고 채널(특징 종류)은 늘어남
- Classification: Flatten으로 한 줄로 편 뒤 MLP로 분류 (이미 배운 부분)

용어 정리: CNN은 어디까지일까

CNN = Convolutional Neural Network (합성곱 신경망) = 합성곱을 핵심 부품으로 쓰는 신경망 전체

```
CNN (모델 전체)
├─ Feature extraction
│    ├─ 합성곱 → ReLU → Pooling     ← 합성곱 블록 1
│    └─ 합성곱 → ReLU → Pooling     ← 합성곱 블록 2
└─ Classification
     └─ Flatten → FC → ReLU → FC
        (→ 시그모이드 / 소프트맥스)
```

용어	가리키는 범위
합성곱	커널을 밀며 곱해 더하는 계산 하나
합성곱층	합성곱 계산을 하는 층 하나 (nn.Conv2d). 그림에서는 ReLU까지 묶어 그리기도 함
합성곱 블록	합성곱 → ReLU → Pooling 묶음
CNN	**합성곱 블록들 + Flatten + FC층까지 모델 전체**

순서는 합성곱 → ReLU → Pooling. 실습 코드의 class CNN 안에 features(합성곱 블록)와 classifier(Flatten + FC층)가 모두 들어 있음

커널(필터) = 학습할 w

w_1	w_2	w_3
w_4	w_5	w_6
w_7	w_8	w_9

3×3 커널 = w 9개 + b 1개

커널 값 = w

처음엔 랜덤 값. loss를 줄이는 방향으로 역전파 + 경사하강법으로 학습함. 사람이 설계하지 않아도 세로선·모서리 검출기 같은 커널이 저절로 만들어짐.

가중치 공유 (weight sharing)

같은 커널을 이미지의 모든 위치에서 재사용. 5×5 입력 → 3×3 출력일 때 FC층은 w 225개, 합성곱은 w 9개 + b 1개.

합성곱 한 칸 계산



주황 칸 계산

0×(−1) + 0×0 + 1×1
+ 0×(−1) + 0×0 + 1×1
+ 0×(−1) + 0×0 + 1×1
= 3

입력의 3×3 영역과 커널을 같은 자리끼리 곱한 뒤 전부 더함 (+ b) → 숫자 1개

커널을 밀며 특징 맵 완성

위치 (0,0) → 0

0	0	0	1	1
0	0	0	1	1
0	0	0	1	1
0	0	0	1	1
0	0	0	1	1

위치 (0,1) → 3

0	0	0	1	1
0	0	0	1	1
0	0	0	1	1
0	0	0	1	1
0	0	0	1	1

위치 (1,2) → 3

0	0	0	1	1
0	0	0	1	1
0	0	0	1	1
0	0	0	1	1
0	0	0	1	1

특징 맵 완성

0	3	3
0	3	3
0	3	3

계산 순서

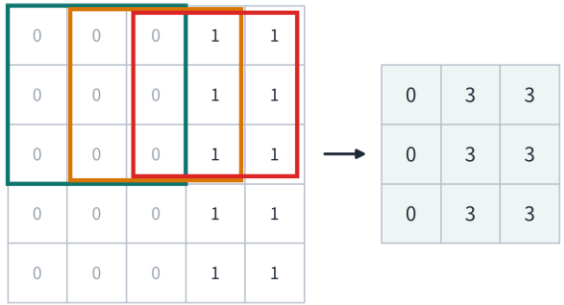
커널을 한 칸씩 오른쪽으로, 줄 끝에서 아래로 이동하며 반복. 결과 모음 = 특징 맵(feature map). 5×5 입력 + 3×3 커널 → 3×3

특징 맵의 의미

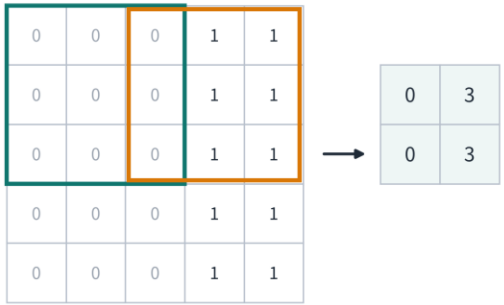
커널 [−1, 0, 1] = 세로 경계 검출기. 0 → 1로 바뀌는 경계 근처에서 3, 값이 같은 곳에서 0. 깊은 층일수록 단순한 특징을 조합한 복잡한 특징을 찾음.

Stride: 커널이 한 번에 이동하는 칸 수

stride 1: 한 번에 1칸 이동 → 출력 3×3



stride 2: 한 번에 2칸 이동 → 출력 2×2



출력 감소

stride 2 → 가로·세로 약 절반

계산 감소

커널을 놓는 위치 수 감소

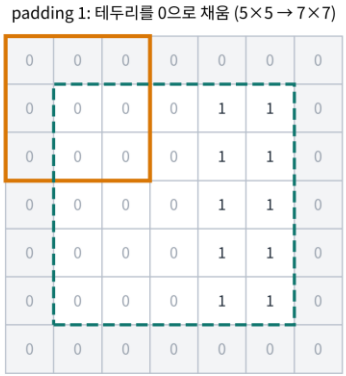
정보 손실

건너뛴 위치의 세밀한 패턴을 놓칠 수 있음

실제 CNN 합성곱층은 대부분 stride 1 사용. 크기 줄이기는 주로 Pooling이 담당

Padding: 테두리를 0으로 채우기





이름	padding	결과	PyTorch
valid	0	크기 줄어듦	`nn.Conv2d(1, 16, 3, padding=0)`
same	3×3 커널이면 1	**크기 유지**	`nn.Conv2d(1, 16, 3, padding=1)`

padding 없이 합성곱하면

① 크기가 계속 줄어듦: 5×5 → 3×3 → 1×1
② 가장자리 손실: 가운데 픽셀 9번, 모서리 1번만 계산에 쓰임 (왼쪽 그림)

zero padding

테두리에 0을 채우면 원래 모서리도 커널 가운데에 올 수 있음 (오른쪽 그림). 5×5 + padding 1 → 7×7 → 합성곱 → 5×5 유지

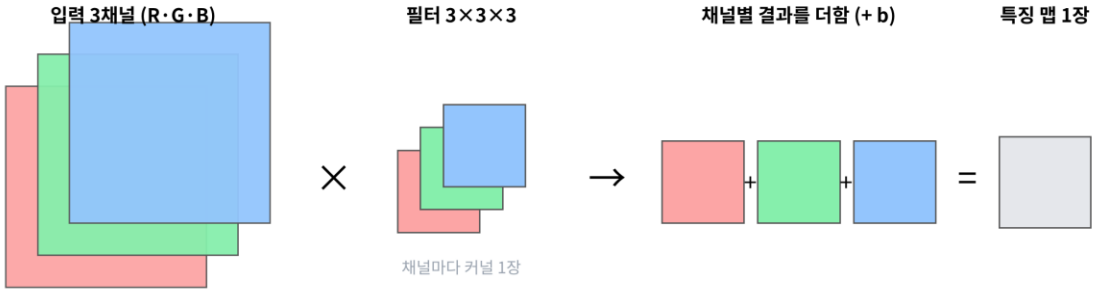
출력 크기 공식

$$\text{출력 크기} = (N + 2P - F) / S + 1$$

N: 입력 크기 P: padding F: 커널 크기 S: stride

예시	N	P	F	S	계산	출력
기본 합성곱	5	0	3	1	(5 + 0 − 3) / 1 + 1	**3**
stride 2	5	0	3	2	(5 + 0 − 3) / 2 + 1	**2**
padding 1	5	1	3	1	(5 + 2 − 3) / 1 + 1	**5**
MNIST 첫 합성곱	28	1	3	1	(28 + 2 − 3) / 1 + 1	**28**
Max Pooling 2×2	28	0	2	2	(28 + 0 − 2) / 2 + 1	**14**

컬러 입력과 여러 필터



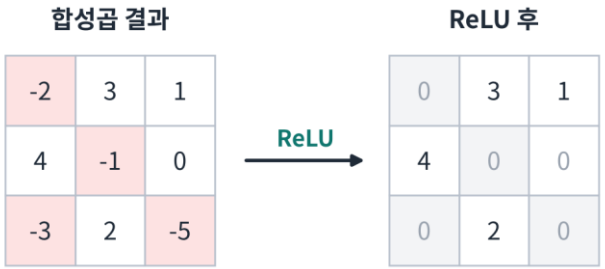
층 (Fashion-MNIST 모델)	계산	파라미터
합성곱 1 (1→16)	(3×3×1 + 1) × 16	160
합성곱 2 (16→32)	(3×3×16 + 1) × 32	4,640
FC 1 (1568→128)	1568 × 128 + 128	**200,832**
FC 2 (128→10)	128 × 10 + 10	1,290
합계		**206,922**

- 필터 1개 = 채널 수만큼의 커널 묶음 (3×3×3 = w 27개 + b 1개)
- 채널별 합성곱 결과를 더해 특징 맵 1장 생성
- 필터 개수 = 출력 채널 수: nn.Conv2d(3, 16, 3) → 입력 3채널, 출력 16채널

비교

합성곱층 w는 커널 크기로만 정해짐 (160개). 다만 이 작은 예제는 FC층 때문에 전체 206,922개로, 같은 크기 MLP(784→128→10, 101,770개)보다 많음. 큰 이미지일수록 CNN이 유리함.

합성곱 뒤에는 ReLU



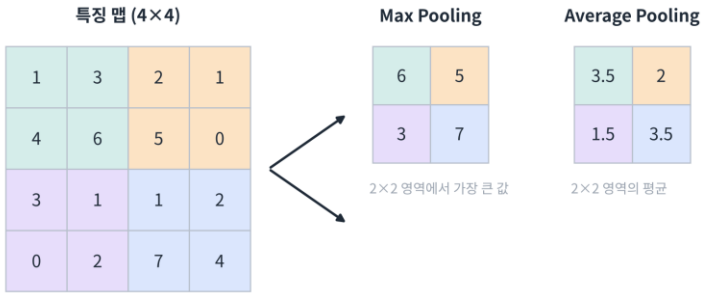

```
nn.Conv2d(1, 16, 3, padding=1),    # 합성곱
nn.ReLU(),                         # 활성화
nn.MaxPool2d(2),                   # Pooling
```

ReLU(x) = max(0, x)

음수는 0, 양수는 그대로. 비선형성을 넣어 층을 쌓는 의미를 만듦.

이름	들어 있는 것
합성곱층 (그림)	합성곱 + ReLU
Pooling층	Pooling만
합성곱 블록	합성곱 + ReLU + Pooling

Pooling: 영역의 대표값만 남기기



계산

Max: max(1,3,4,6) = 6, max(2,1,5,0) = 5
Average: (1+3+4+6)/4 = 3.5, (2+1+5+0)/4 = 2
CNN에서는 주로 Max Pooling 사용

학습할 w 없음

최댓값·평균을 고르는 정해진 계산

크기 감소

2×2면 가로·세로 절반, 계산량 1/4

위치 변화에 강함

특징이 한두 칸 움직여도 최댓값 유지

과적합 완화

세밀한 값을 버려 외우기 방지

참고

Pooling으로 크기가 1/4이 되면 다음 합성곱층에서 필터 수를 늘려(16 → 32) 정보량을 보충하는 경우가 많음.

크기 추적표와 Flatten

단계	코드	출력 크기	부분
입력		1 × 28 × 28	
합성곱 + ReLU	`Conv2d(1, 16, 3, padding=1)`	16 × 28 × 28	**Feature extraction**
Max Pooling	`MaxPool2d(2)`	16 × 14 × 14	**Feature extraction**
합성곱 + ReLU	`Conv2d(16, 32, 3, padding=1)`	32 × 14 × 14	**Feature extraction**
Max Pooling	`MaxPool2d(2)`	32 × 7 × 7	**Feature extraction**
Flatten	`Flatten()`	1568	모양만 바꿈
FC + ReLU	`Linear(1568, 128)`	128	**Classification**
출력	`Linear(128, 10)`	10	**Classification**

Flatten

7×7×32 상자를 1568개짜리 한 줄로 폄. 계산 없이 모양만 바꾸는 층. 위치 정보는 이미 특징 안에 담겨 있음.

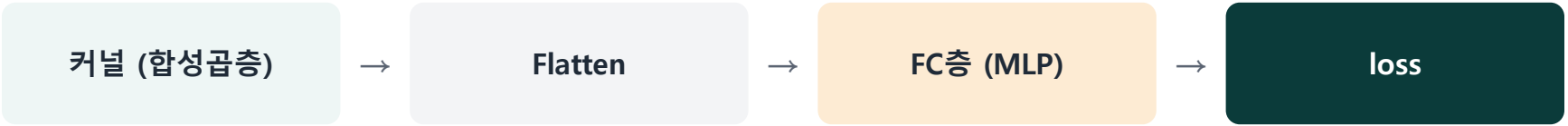
Classification = MLP

역전파 예제의 '입력 2 → 은닉 2 → 출력 1'과 같은 구조. 입력만 날씨 숫자 2개에서 CNN 특징 1568개로 바뀜.

이진 분류 vs 다중 분류

	이진 분류 (실습 ①)	다중 분류 (실습 ②)	회귀 (참고)
예	고양이 / 개	옷 10종류	기온 예측
출력 뉴런 수	1	클래스 수 (10)	1
은닉층 활성화	ReLU	ReLU	ReLU
출력층 활성화	**시그모이드**	**소프트맥스**	없음
손실 함수	**BCE**	**크로스엔트로피**	MSE
PyTorch 손실	`nn.BCEWithLogitsLoss()`	`nn.CrossEntropyLoss()`	`nn.MSELoss()`
정답 형식	0.0 / 1.0 (실수)	0 ~ 9 (정수)	실수
예측 읽는 법	sigmoid(out) ≥ 0.5 → 1	out.argmax(1)	out 그대로

CNN도 같은 방식으로 학습 (end-to-end)



← 역전파: loss의 미분값이 FC층과 Flatten을 지나 커널까지 전달됨

학습 단계	PyTorch 코드	앞에서 배운 것
1. 순전파	`out = model(X)`	합성곱·Pooling·FC 계산
2. 손실 계산	`loss = loss_fn(out, y)`	BCE / 크로스엔트로피
3. 역전파	`loss.backward()`	체인룰로 커널과 w의 미분
4. 업데이트	`optimizer.step()`	Adam 옵티마이저 (경사하강법 계열)

옵티마이저: 경사하강법 계열 정리

모두 기본은 w ← w − lr × 기울기. 얼마나, 어떤 방향으로 움직일지 정하는 방식만 다름

옵티마이저	기울기 계산에 쓰는 데이터	특징
경사하강법 (배치)	전체 데이터	날씨 예제처럼 7개 평균. 데이터가 많으면 느림
SGD	데이터 일부 (미니배치)	자주 업데이트해서 빠름. 실습의 batch_size=64가 미니배치
모멘텀	미니배치	SGD + 관성 (이전 이동 방향을 기억)
RMSProp	미니배치	SGD + w마다 다른 학습률
Adam	미니배치	**모멘텀 + RMSProp. 실습에서 사용 (lr = 0.001)**

```
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)    # Adam
optimizer = torch.optim.SGD(model.parameters(), lr=0.01, momentum=0.9)    # SGD + 모멘텀
```

데이터 나누기: train / validation / test

학습 (train)

w를 업데이트하는 데이터. 교과서 연습 문제처럼 에폭마다 반복해서 풂

검증 (validation)

에폭마다 성적만 확인, w 업데이트 안 함. 모의고사처럼 가장 좋은 시점과 하이퍼파라미터를 고르는 데 사용

테스트 (test)

학습이 다 끝난 뒤 딱 한 번 사용. 수능처럼 처음 보는 데이터로 최종 성적 확인

구분	실습 ① 고양이·개	실습 ② Fashion-MNIST	w 업데이트	batch_size	shuffle
train	8,000장	50,000장	**O**	**64**	`True`
validation	2,000장	10,000장	X	256	`False`
test	2,000장	10,000장	X	256	`False`

val·test는 w를 고치지 않고 계산만 함 → 배치를 크게(256) 잡아 빨리 평가, 순서는 섞을 필요 없음

```
train_data, val_data = random_split(train_full, [50000, 10000])   # 원래 학습 데이터에서 검증용을 떼어 냄
train_loader = DataLoader(train_data, batch_size=64, shuffle=True)
val_loader   = DataLoader(val_data,   batch_size=256)
test_loader  = DataLoader(test_data,  batch_size=256)   # 테스트는 처음부터 따로 제공됨
```

배치와 에폭, shuffle

train 50,000장 → 64장씩 묶음 (배치) → 배치 782개 → 782번 업데이트 = 1 에폭

배치 1개마다 순전파 → loss → 역전파 → w 업데이트 1번. 실습 ②는 5 에폭 → w를 약 3,910번 고침 (마지막 배치는 16장)

batch_size	1 에폭 업데이트 횟수	특징
1	50,000번	자주 고치지만 방향이 많이 흔들림
64 (실습)	**782번**	**둘의 절충. 실제로 가장 많이 씀**
50,000 (전체)	1번	방향은 정확, 느리고 메모리 많이 씀

shuffle=True: 에폭마다 순서 섞기

데이터가 고양이 5,000장 → 개 5,000장 순서로 정렬돼 있으면 앞 배치는 고양이만, 뒤 배치는 개만 보게 되어 w가 한쪽으로 쏠림. 섞으면 배치마다 고르게 섞이고, 에폭마다 배치 구성도 달라짐.

```
for epoch in range(EPOCHS):                  # 에폭 반복
    for X, y in train_loader:                # 배치 반복 (64장씩, 에폭마다 새로 섞임)
        loss = loss_fn(model(X), y)
        optimizer.zero_grad(); loss.backward(); optimizer.step()    # 배치마다 w 1번 업데이트
```

가장 좋은 모델 고르기

epoch	학습 loss	검증 loss	
10	0.40	0.45	
20	0.20	**0.30**	**가장 좋음 → 저장**
30	0.10	0.35	과적합 시작

예시 숫자. 학습 loss는 계속 줄어도 검증 loss는 다시 나빠질 수 있음

```
if val_loss < best_val_loss:
    best_val_loss = val_loss
    torch.save(model.state_dict(), SAVE_PATH)

# 학습 후 가장 좋았던 w로 테스트
model.load_state_dict(torch.load(SAVE_PATH))
```

조기 종료 (early stopping)

검증 loss가 몇 번 연속 좋아지지 않으면 학습 중단, 저장해 둔 가장 좋은 w 사용.

실습 ① 데이터: CIFAR-10에서 고양이·개만

```
transform = transforms.ToTensor()    # 텐서로 바꾸고 0~1로 정규화
cifar_train = datasets.CIFAR10(root="data", train=True,
                               download=True, transform=transform)
CAT, DOG = 3, 5

class CatDog(Dataset):
    def __init__(self, cifar):
        self.cifar = cifar
        self.idx = [i for i, t in enumerate(cifar.targets) if t in (CAT, DOG)]
    def __len__(self):
        return len(self.idx)
    def __getitem__(self, i):
        x, t = self.cifar[self.idx[i]]
        return x, torch.tensor(1.0 if t == DOG else 0.0)

train_full = CatDog(cifar_train)
train_data, val_data = random_split(train_full, [8000, 2000])
train_loader = DataLoader(train_data, batch_size=64, shuffle=True)
```

고양이 = 0, 개 = 1

CIFAR-10 10종류 중 3번(고양이), 5번(개)만 사용. 학습 10,000장(검증 2,000장 포함), 테스트 2,000장.

정답을 실수로

BCE 손실은 정답을 0.0 / 1.0 실수로 받음 → torch.tensor(1.0).

실습 ① 모델과 학습

```
class CNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 16, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(16, 32, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2))
        self.classifier = nn.Sequential(
            nn.Flatten(), nn.Linear(32 * 8 * 8, 128), nn.ReLU(),
            nn.Linear(128, 1))                          # 출력 1개
    def forward(self, x):
        return self.classifier(self.features(x)).squeeze(1)

loss_fn = nn.BCEWithLogitsLoss()                # 시그모이드 + BCE
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

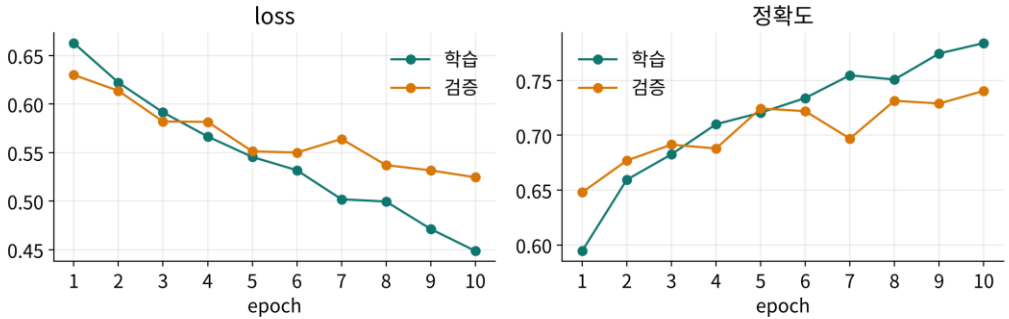
for epoch in range(1, 11):
    for X, y in train_loader:
        out = model(X)                          # 1. 순전파
        loss = loss_fn(out, y)                  # 2. 손실 계산
        optimizer.zero_grad()                   #    이전 미분값 초기화
        loss.backward()                         # 3. 역전파
        optimizer.step()                        # 4. 업데이트
```

층	크기
입력	3×32×32
합성곱+ReLU	16×32×32
Pooling	16×16×16
합성곱+ReLU	32×16×16
Pooling	32×8×8
Flatten	2048
FC	128 → 1

BCEWithLogitsLoss

모델은 시그모이드 전 값(logit)을 출력, 손실 함수가 시그모이드 + BCE를 한 번에 계산 → 숫자 계산이 더 안정적. 파라미터 267,489개.

실습 ① 결과: 고양이 vs 개



테스트 정확도

72.9%

검증 정확도

74.1%

해석

학습 78% > 검증 74%로 차이가 벌어지기 시작. 7 epoch에서 검증 정확도 잠깐 하락 → 검증 loss가 가장 낮은 w 저장. 32×32 작은 사진이라 어려운 문제.



개선 방법

epoch 늘리기 · 합성곱 블록 추가 · 데이터 증강 · 사전 학습 모델

실습 ② 데이터와 모델: Fashion-MNIST

```
train_full = datasets.FashionMNIST(root="data", train=True,
                                   download=True, transform=transforms.ToTensor())
train_data, val_data = random_split(train_full, [50000, 10000])

class CNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 16, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(16, 32, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2))
        self.classifier = nn.Sequential(
            nn.Flatten(), nn.Linear(32 * 7 * 7, 128), nn.ReLU(),
            nn.Linear(128, 10))                       # 출력 10개
    def forward(self, x):
        return self.classifier(self.features(x))

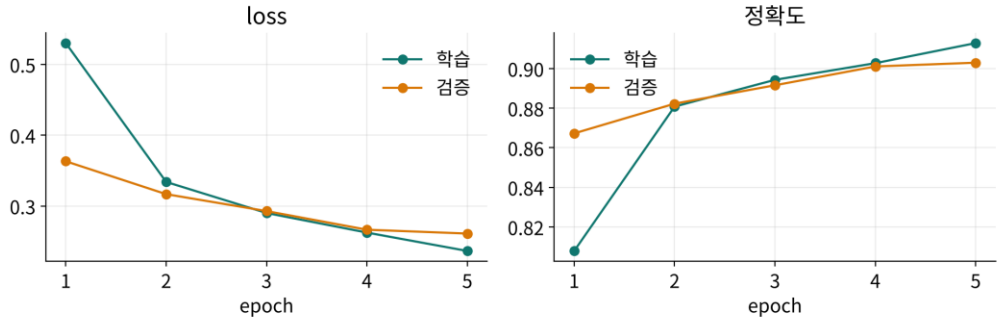
loss_fn = nn.CrossEntropyLoss()                 # 소프트맥스 + 크로스엔트로피
def predict(out):
    return out.argmax(dim=1)                    # 가장 큰 값의 클래스
```

바뀐 곳	①	②
입력 채널	3	1
Flatten	32×8×8	32×7×7
출력 수	1	10
손실	BCE	CE
예측	≥ 0.5	argmax

10종류, 28×28 흑백

학습 60,000장(검증 10,000장 포함), 테스트 10,000장. 합성곱 블록 구조는 실습 ①과 동일.

실습 ② 결과: 옷 10종류



많이 헷갈린 조합: 정답 → 예측	개수
Shirt (셔츠) → T-shirt/top (티셔츠)	**123**
Shirt (셔츠) → Coat (코트)	92
Shirt (셔츠) → Pullover (스웨터)	79
Pullover (스웨터) → Coat (코트)	75

테스트 정확도

90.2%

검증 정확도

90.3%

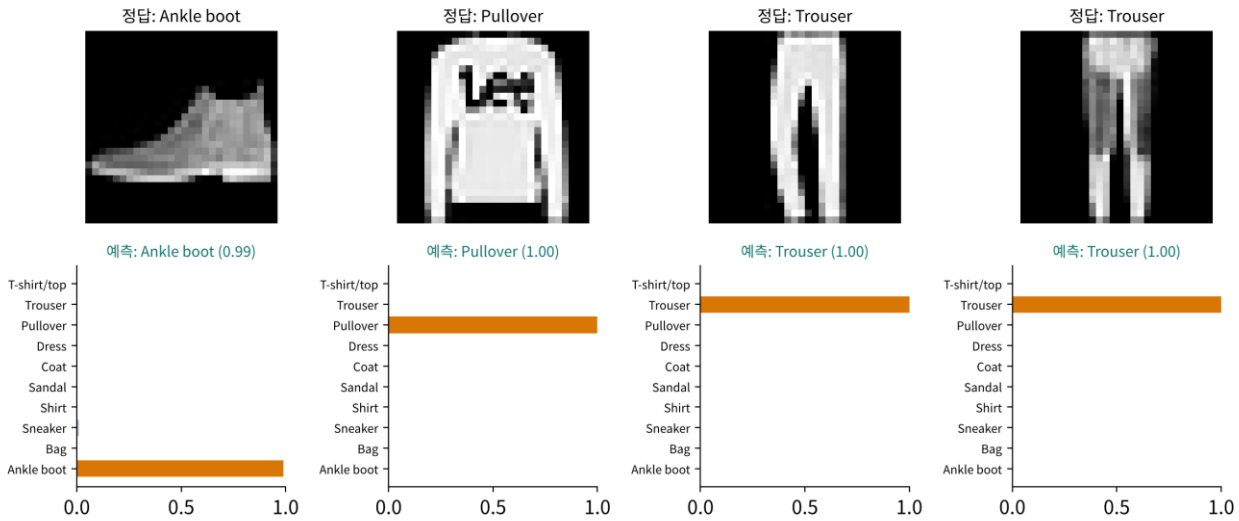
해석

검증 정확도 86.7% → 90.3%. 학습·검증 loss가 함께 감소 → 과적합 없음. 무작위로 고르면 10%인 문제를 작은 CNN으로 90% 정답.

왜 셔츠?

28×28 흑백에서 셔츠·티셔츠·스웨터·코트는 비슷한 상의 모양. 사람도 헷갈리는 조합.

실습 ② 예측 확률 보기



읽는 법

막대 길이 = 각 클래스의 확률. 4장 모두 0.99 이상 확신하고 정답.

혼동 행렬

정답 클래스(행)별로 무엇으로 예측했는지 센 10×10 표. 노트북에서 직접 계산.

문제 1·3: 출력 크기, 출력층과 손실 함수

문제 1. 출력 크기 계산 (힌트: (N + 2P − F) / S + 1)

	입력 N	padding P	커널 F	stride S	출력
(1)	32	0	5	1	?
(2)	32	2	5	1	?
(3)	7	0	3	2	?
(4)	28 (Max Pooling 2×2)	0	2	2	?

문제 3. 출력 뉴런 수, 출력층 활성화, PyTorch 손실 함수 고르기

문제	출력 뉴런 수	출력층 활성화	PyTorch 손실 함수
(1) 이메일이 스팸인지 아닌지	?	?	?
(2) 손글씨 숫자 0~9 맞히기	?	?	?
(3) 사진으로 내일 기온 예측	?	?	?
(4) 사진 속 동물 5종류 맞히기	?	?	?

문제 2: 합성곱과 Max Pooling 손으로 계산

입력 (4×4)

1	2	0	1
0	1	3	1
2	0	1	0
1	1	0	2

커널 (3×3)

1	0	1
0	1	0
1	0	1

커널에서 1인 자리 5곳(네 모서리 + 가운데)의 입력값을 더하면 됨

조건: padding 0, stride 1, b = 0

(1) 특징 맵의 크기는?
(2) 특징 맵의 값 4개는?
(3) ReLU 적용 후 2×2 Max Pooling(stride 2) 결과는?
(4) Max 대신 Average Pooling이면?

정답

문제 1

(1) (32 − 5)/1 + 1 = 28
(2) (32 + 4 − 5)/1 + 1 = 32
(3) (7 − 3)/2 + 1 = 3
(4) (28 − 2)/2 + 1 = 14

문제 2

(1) (4 − 3)/1 + 1 = 2 → 2×2
(2) 왼쪽 위 1+0+1+2+1 = 5
오른쪽 위 2+1+3+0+0 = 6
왼쪽 아래 0+3+0+1+0 = 4
오른쪽 아래 1+1+1+1+2 = 6
→ [[5, 6], [4, 6]]
(3) 모두 양수라 ReLU 후 그대로, max = 6
(4) (5 + 6 + 4 + 6)/4 = 5.25

문제 3

(1) 1 · 시그모이드 · BCEWithLogitsLoss
(2) 10 · 소프트맥스 · CrossEntropyLoss
(3) 1 · 없음 · MSELoss (회귀)
(4) 5 · 소프트맥스 · CrossEntropyLoss

정리

01	**합성곱**	커널(= w)을 밀며 칸끼리 곱해 더하기, 같은 커널 재사용
02	**Stride · Padding**	이동 칸 수와 테두리 0 채우기. 출력 = (N + 2P − F) / S + 1
03	**ReLU · Pooling**	음수는 0, Max Pooling으로 크기 절반 (w 없음)
04	**전체 구조**	Feature extraction → Flatten → MLP 분류, 역전파로 한 번에 학습
05	**출력층**	이진: 1개 + 시그모이드 + BCE · 다중: N개 + 소프트맥스 + 크로스엔트로피
06	**학습**	미니배치 + Adam으로 업데이트, val로 가장 좋은 w 선택, test로 최종 확인
07	**참고 자료**	rubber-tree.tistory.com/116 · CIFAR-10 · Fashion-MNIST (그림·결과 직접 제작)